

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



DEPARTMENT OF COMPUTER NETWORKS AND TELECOMMUNICATION

REPORT OF PROJECT

Project: Video Streaming System using RTSP/RTP protocols

Lecture:	Lê Hà Minh	
Leader:	Khang Phuc Nguyen	24120068
Members:	Nghia Trong Hoang	24120103
	Trong Minh Pham	24120151

Ho Chi Minh City, 12/2025

Table of Contents



Giới thiệu

Với sự phát triển của công nghệ số và nhu cầu giải trí của con người tăng cao, đặc biệt khi cuộc cách mạng 4.0, sự bùng nổ của Internet thì vai trò của Mạng Máy Tính trở nên ngày càng quan trọng và có nhiều ứng dụng thực tiễn chẳng hạn là như là nhắn tin trực tuyến, làm việc trực tuyến và đặc biệt là trao đổi các thông tin, dữ liệu thông quan mạng. Đặc biệt trong số đó thì việc truyền các Video hoặc Ảnh thông qua được sử dụng ngày phổ biến trong các ứng dụng mạng xã hội như Facebook, TikTok và Youtube.

Trong dự án này, nhóm đã tìm hiểu và thực hiện hóa một hệ thống đơn giản cho phép hai ứng dụng mạng có thể giao tiếp với nhau thông qua giao tiếp RTSP/RTP để có thể phát một video trực tuyến. Dự án này được làm với mục đích học tập và hiểu rõ hơn về các giao thức cũng như là cách hoạt động của các ứng dụng trong thực tế, mặt khác, dự án này cũng chỉ là mô phỏng lại cách phát trực tiếp một video giữa các ứng dụng nên được xây dựng không quá phức tạp và tối ưu.

Toàn bộ mã nguồn của dự án được xây dựng bằng ngôn ngữ `C++ 17`, sử dụng `CMake` để có thể build trên cả Window và Linux, cũng như là sử dụng các thư viện `winsock2` và `POSIX sockets` để tạo các socket cho việc giao tiếp giữa các ứng dụng. Bên cạnh đó, nhóm cũng đã áp dụng các kiến thức về `Design Pattern` cho dự án với mục đích là mã nguồn của dự án có thể tái sử dụng, sạch sẽ và dễ đọc, dễ sửa lỗi cũng như là tối ưu trong quá trình hoạt động.

Để có thể có một giao diện cho ứng dụng của bên phía `client` thì nhóm đã sử dụng `Qt6` để tạo ra một giao diện đơn giản và dễ sử dụng cho người dùng. Toàn bộ quá trình, giao thức, phản hồi và yêu cầu của `server` và `client` được dựa trên toàn bộ yêu cầu được đính kèm trong file `Socket_2526` của thầy Lê Hà Minh [?]. Nhóm xin chân thành cảm ơn thầy đã hỗ trợ và cung cấp tài liệu để nhóm có thể hoàn thành đồ án này một cách tốt đẹp nhất.

Nhóm Trưởng

(Chữ ký và họ tên)

Khang

Nguyễn Phúc Khang

Ho Chi Minh City, 12/2025

1 Cơ sở lý thuyết

Hệ thống truyền phát video trong đề án được xây dựng dựa trên sự kết hợp chặt chẽ giữa kiến trúc mạng máy tính kinh điển và các kỹ thuật xử lý dữ liệu đa phương tiện thời gian thực, tuân thủ theo các yêu cầu của môn học [?]. Phần này sẽ trình bày chi tiết về các giao thức, kỹ thuật socket nâng cao và các mẫu thiết kế phần mềm được áp dụng.

1.1 Mô hình kiến trúc và Giao diện Socket

Hệ thống vận hành dựa trên mô hình **Client-Server**, trong đó trách nhiệm được phân chia rõ ràng giữa hai thực thể. **Server** đóng vai trò là bên cung cấp dịch vụ thụ động (*Passive Open*), chịu trách nhiệm lưu trữ tài nguyên video, quản lý các phiên làm việc và điều phối luồng dữ liệu. Ngược lại, **Client** đóng vai trò chủ động (*Active Open*), khởi tạo các yêu cầu kết nối và thực hiện tái tạo hình ảnh từ dữ liệu nhận được.

Để hiện thực hóa cơ chế giao tiếp này, dự án sử dụng **Socket API** như một giao diện lập trình cốt lõi [?]. Socket đóng vai trò là điểm cuối trong liên kết hai chiều giữa hai chương trình chạy trên mạng. Tùy thuộc vào yêu cầu về độ tin cậy hay tốc độ của từng loại dữ liệu, hệ thống sẽ khởi tạo loại socket phù hợp: **Stream Socket** (sử dụng TCP) cho kênh điều khiển nhằm đảm bảo tính chính xác tuyệt đối của lệnh, hoặc **Datagram Socket** (sử dụng UDP) cho kênh dữ liệu để tối ưu hóa độ trễ thời gian thực. Bên cạnh đó, các tùy chọn nâng cao như `SO_REUSEADDR` cũng được áp dụng để quản lý trạng thái cổng kết nối hiệu quả, tránh các lỗi khóa tài nguyên thường gặp trong lập trình mạng UNIX [?].

1.2 Các giao thức tầng giao vận

Việc lựa chọn giao thức tầng giao vận là quyết định kỹ thuật quan trọng nhất ảnh hưởng đến hiệu năng của ứng dụng streaming. Thay vì chỉ sử dụng một giao thức đơn lẻ, hệ thống áp dụng cơ chế lai (Hybrid) để tận dụng ưu điểm của cả TCP và UDP theo kiến trúc nền tảng về mạng máy tính [?]. Sự khác biệt và phạm vi áp dụng của hai giao thức này trong dự án được tóm tắt tại Bảng ??.

Table 1: So sánh và phạm vi áp dụng của TCP và UDP trong hệ thống

Giao thức	Đặc điểm kỹ thuật	Áp dụng trong dự án
TCP	Hướng kết nối, đảm bảo tin cậy, có kiểm soát luồng và tắc nghẽn, độ trễ cao hơn do cơ chế truyền lại (retransmission).	Kênh RTSP: Truyền tải các lệnh điều khiển quan trọng như SETUP, PLAY.
UDP	Phi kết nối, không đảm bảo tin cậy, header nhỏ (8 byte), độ trễ thấp, hỗ trợ truyền thời gian thực.	Kênh RTP: Vận chuyển dữ liệu video (JPEG). Chấp nhận mất gói để giữ độ trễ thấp.

1.3 Giao thức điều khiển RTSP

Real-Time Streaming Protocol (RTSP), được định nghĩa trong RFC 2326 [?], hoạt động như một "bộ điều khiển từ xa" qua mạng cho các máy chủ đa phương tiện. Điểm khác biệt cốt lõi của RTSP so với HTTP là tính chất *lưu trạng thái* (stateful). Server duy trì một phiên làm việc thông qua mã định danh **Session ID**, cho phép Server theo dõi trạng thái hiện tại của Client để phản hồi hợp lý.

Quá trình chuyển đổi trạng thái của hệ thống tuân theo quy trình nghiêm ngặt: từ trạng thái **INIT** (khởi tạo), chuyển sang **READY** (sẵn sàng) sau khi thiết lập tài nguyên thành công, và cuối cùng là **PLAYING** (đang phát) khi luồng dữ liệu bắt đầu được truyền tải. Các phương thức điều khiển chính được mô tả trong Bảng ??.

Table 2: Các phương thức RTSP và chức năng tương ứng

Phương thức	Mô tả chức năng
SETUP	Khởi tạo phiên làm việc. Client gửi cổng nhận RTP (<i>client_port</i>), Server phản hồi với <i>Session ID</i> và cổng truyền của Server.
PLAY	Yêu cầu Server bắt đầu truyền dữ liệu RTP. Server chuyển trạng thái sang PLAYING.
PAUSE	Tạm dừng luồng dữ liệu. Server ngừng gửi gói tin nhưng vẫn giữ nguyên phiên làm việc và trạng thái kết nối.
TEARDOWN	Kết thúc phiên làm việc. Server đóng kết nối, giải phóng bộ đệm và thu hồi Session ID.

1.4 Giao thức vận chuyển RTP và Cấu trúc gói tin

Trong khi RTSP chịu trách nhiệm điều khiển, **Real-time Transport Protocol (RTP)** theo tiêu chuẩn RFC 3550 [?] đảm nhiệm vai trò vận chuyển dữ liệu video thực tế. Do đặc thù của giao thức UDP là không đảm bảo thứ tự, RTP bổ sung một phần tiêu đề (Header) dài 12 byte vào trước mỗi gói dữ liệu để cung cấp thông tin tái đồng bộ.

Ba trường thông tin quan trọng nhất trong RTP Header bao gồm: **Sequence Number** (16 bit) giúp bên nhận phát hiện mất gói và sắp xếp lại các gói tin bị đảo lộn; **Timestamp** (32 bit) đóng vai trò nhận thời gian để đồng bộ hóa tốc độ hiển thị khung hình, đảm bảo video không bị tua nhanh hoặc chậm bất thường; và **Marker Bit** (1 bit) dùng để đánh dấu gói tin cuối cùng của một khung hình, hỗ trợ quá trình tái lắp ráp dữ liệu phân mảnh.

1.5 Kỹ thuật Video Streaming và Xử lý bộ đệm

Để đảm bảo khả năng truyền tải video mượt mà trên môi trường mạng biến động, hệ thống áp dụng các kỹ thuật xử lý dữ liệu nâng cao liên quan đến định dạng nén, phân mảnh và cơ chế đệm.

1.5.1 Định dạng MJPEG và Phân mảnh gói tin (Fragmentation)

Dự án sử dụng chuẩn nén **MJPEG (Motion JPEG)**, nơi mỗi khung hình video là một bức ảnh JPEG độc lập sử dụng kỹ thuật nén nội khung (intra-frame compression) [?]. Phương pháp này tuân thủ RFC 2435 [?] và mang lại khả năng chịu lỗi cao, vì việc mất một gói tin chỉ ảnh hưởng cục bộ đến khung hình hiện tại mà không gây lỗi dây chuyền cho các khung hình kế tiếp.

Tuy nhiên, kích thước của một khung hình video (đặc biệt ở độ phân giải HD) thường vượt quá giới hạn **MTU (Maximum Transmission Unit)** của mạng Ethernet (thông thường là 1500 byte). Để giải quyết vấn đề này, dự án áp dụng kỹ thuật **Phân mảnh cấp ứng dụng** (Application-Level Fragmentation). Một khung hình lớn sẽ được chia nhỏ thành nhiều gói RTP có kích thước tối đa 1400 byte. Hệ thống sử dụng mẫu thiết kế **Strategy Pattern** [?] để linh hoạt chuyển đổi thuật toán xử lý giữa video SD (có thể không cần phân mảnh) và video HD (bắt buộc phân mảnh), đảm bảo tính tương thích và hiệu năng cao nhất.

1.5.2 Cơ chế Bộ đệm (Buffering) và Kiểm soát Jitter

Hiện tượng **Jitter** (sự biến thiên độ trễ gói tin) là nguyên nhân chính gây ra tình trạng giật hình trong streaming [?]. Để khắc phục, Client được thiết kế hoạt động theo mô hình **Producer-**

Consumer với một **Frame Buffer** trung gian. Luồng nhận mạng đóng vai trò Producer, liên tục tái lắp ráp các gói tin RTP và đẩy khung hình hoàn chỉnh vào hàng đợi. Luồng hiển thị giao diện đóng vai trò Consumer, lấy khung hình ra để hiển thị theo định kỳ.

Đặc biệt, kỹ thuật **Pre-buffering** (Tiền tải) được áp dụng nhằm yêu cầu Client phải tích lũy đủ một lượng khung hình nhất định (ngưỡng N frames) trước khi bắt đầu phát. Cơ chế này giúp tạo ra một vùng đệm an toàn, triệt tiêu các dao động tức thời của mạng và đảm bảo trải nghiệm xem video liên tục.

1.6 Các mẫu thiết kế phần mềm (Design Patterns)

Để đảm bảo mã nguồn có khả năng mở rộng, dễ bảo trì và tuân thủ các nguyên lý lập trình hướng đối tượng, dự án đã áp dụng một số mẫu thiết kế phần mềm kinh điển được đề xuất bởi Gamma và cộng sự [?]. Việc áp dụng các mẫu này giúp giải quyết các vấn đề phổ biến trong việc khởi tạo đối tượng, quản lý trạng thái toàn cục và đơn giản hóa các giao diện phức tạp.

1.6.1 Mẫu thiết kế Builder (Builder Pattern)

Trong giao thức RTP, việc khởi tạo một gói tin (**RTPPacket**) đòi hỏi thiết lập rất nhiều tham số chi tiết cho phần tiêu đề (Header) như: phiên bản (Version), chỉ số đếm (Sequence Number), nhãn thời gian (Timestamp), bit đánh dấu (Marker), và loại tải (Payload Type). Việc sử dụng một hàm khởi tạo (Constructor) với quá nhiều tham số sẽ khiến mã nguồn khó đọc và dễ gây lỗi.

Để giải quyết vấn đề này, dự án áp dụng mẫu thiết kế **Builder Pattern**. Cụ thể, lớp **RTPPacketBuilder** (nằm trong thư mục `common/src/rtp`) được xây dựng để tách biệt quá trình khởi tạo phức tạp khỏi biểu diễn của đối tượng. Mẫu thiết kế này cho phép thiết lập từng thuộc tính của gói tin một cách tuần tự và rõ ràng thông qua chuỗi các phương thức (method chaining) trước khi gọi phương thức `build()` để tạo ra đối tượng gói tin hoàn chỉnh.

1.6.2 Mẫu thiết kế Singleton (Singleton Pattern)

Trong một hệ thống phân tán, có những thành phần cần đảm bảo tính duy nhất và có thể truy cập toàn cục từ bất kỳ đâu trong chương trình, điển hình là bộ ghi nhật ký (Logger) và bộ quản lý cấu hình (Configuration Manager).

Dự án áp dụng mẫu thiết kế **Singleton Pattern** cho các lớp **Logger** và **Config** (trong thư mục `common/src/utils`). Mẫu thiết kế này đảm bảo rằng chỉ có duy nhất một thể hiện (instance)

của lớp **Logger** được tạo ra trong suốt vòng đời ứng dụng, giúp đồng bộ hóa việc ghi log từ nhiều luồng (thread) khác nhau vào một tệp tin hoặc màn hình console duy nhất, tránh tình trạng xung đột tài nguyên. Tương tự, **Config** đóng vai trò là kho chứa thông tin cấu hình trung tâm (như địa chỉ IP, cổng, kích thước bộ đệm) mà mọi thành phần khác đều có thể truy xuất.

1.6.3 Mẫu thiết kế Facade (Facade Pattern)

Lập trình mạng trên hệ điều hành thường yêu cầu tương tác với các API cấp thấp phức tạp của thư viện Berkeley Sockets (như `sockaddr_in`, `bind`, `listen`, `setsockopt`). Việc gọi trực tiếp các hàm C này rải rác trong mã nguồn sẽ làm giảm tính trừu tượng và khó chuyển đổi giữa các nền tảng (như Windows và Linux).

Để khắc phục, lớp **Socket** (trong thư mục `common/src/network`) được thiết kế như một **Facade** (mặt tiền). Lớp này cung cấp một giao diện lập trình hướng đối tượng đơn giản và thống nhất (với các phương thức như `send()`, `recv()`, `bind()`) để che giấu sự phức tạp của các lời gọi hệ thống bên dưới. Nhờ đó, các tầng cao hơn của ứng dụng (như RTSP Server hay RTP Sender) chỉ cần làm việc với đối tượng **Socket** mà không cần quan tâm đến chi tiết cài đặt của hệ điều hành.

Table 3: Tổng hợp các mẫu thiết kế được áp dụng trong cấu trúc mã nguồn

Mẫu thiết kế	Loại (Category)	Áp dụng thực tế trong dự án
Builder	Creational (Khởi tạo)	RTPPacketBuilder : Xây dựng gói tin RTP phức tạp từng bước một.
Singleton	Creational (Khởi tạo)	Logger , Config : Quản lý tài nguyên duy nhất (log file, cấu hình) toàn cục.
Facade	Structural (Cấu trúc)	Socket : Đóng gói và đơn giản hóa các thao tác với thư viện socket cấp thấp của hệ điều hành.
Strategy	Behavioral (Hành vi)	EncodingStrategy : Linh hoạt thay đổi thuật toán phân mảnh cho video SD và HD (đã đề cập ở phần trước).

2 Implementation

Mục tiêu của phần này là mô tả đầy đủ cách hệ thống được hiện thực ở mức mã nguồn: cách tổ chức module, cấu trúc lớp, luồng điều khiển của RTSP/RTP, cơ chế đa luồng, thuật toán đóng gói–giải gói RTP (bao gồm phân mảnh cho frame lớn), thuật toán trích xuất frame MJPEG, và cơ chế “buffer như một lớp cache” ở phía Client để ổn định trải nghiệm xem.

2.1 Tổ chức codebase và quy ước build

Dự án được tách thành ba khối chính: **Server**, **Client** và **Common**. Common đóng vai trò thư viện dùng chung cho cả hai phía (network wrapper, RTSP message parsing/building, RTP packet, tiện ích). Việc chia tách này giúp cô lập phụ thuộc (đặc biệt là Qt chỉ xuất hiện ở Client), giảm vòng lặp biên dịch, và làm rõ ranh giới trách nhiệm.

Table 4: Phân rã module và phạm vi trách nhiệm

Module	Trách nhiệm chính	Ví dụ file trọng điểm
<code>common/</code>	Thư viện dùng chung (Socket, RTSP, RTP, Logger/Timer/Config)	<code>common/src/network/Socket.cpp</code> , <code>common/src/rtp/RTPPacket.cpp</code>
<code>server/</code>	RTSP server + luồng streaming RTP + đọc MJPEG	<code>server/src/network/RTSPServer.cpp</code> , <code>server/src/network/ServerWorker.cpp</code> , <code>server/src/video/VideoStream.cpp</code>
<code>client/</code>	RTSP client + nhận RTP + ghép frame + buffer/cache + Qt UI	<code>client/src/network/RTSPClient.cpp</code> , <code>client/src/rtp/RTPReceiver.cpp</code> , <code>client/src/buffer/FrameBuffer.cpp</code> , <code>client/src/ui/ClientUI.cpp</code>

Build system sử dụng CMake và tách các target theo thư mục con; Qt6 chỉ được yêu cầu ở cấp root để build Client. Các include directory được cấu hình để cả Server/Client truy cập được Common.

```
1 find_package(Qt6 REQUIRED COMPONENTS Core Widgets Network Multimedia MultimediaWidgets)
2
3 include_directories(
4     ${CMAKE_SOURCE_DIR}/common/include
5     ${CMAKE_SOURCE_DIR}/client/include
6     ${CMAKE_SOURCE_DIR}/server/include
7 )
```

```
8
9 add_subdirectory(common)
10 add_subdirectory(client)
11 add_subdirectory(server)
```

Listing 1: CMake cấu hình Qt6 và chia subdirectory

2.2 Common layer: chuẩn hóa giao tiếp và tiện ích dùng chung

2.2.1 Socket wrapper và mô hình lỗi

Thay vì gọi trực tiếp POSIX/WinSock API, hệ thống xây dựng lớp `Socket` như một “adapter” thống nhất cho TCP/UDP. Lớp này đóng gói các thao tác then chốt: `bind()/listen()/accept()` cho TCP server, `connect()` cho TCP client, `send()/recv()` cho RTSP qua TCP, và `sendTo()/recvFrom()` cho RTP qua UDP. Điểm quan trọng là lớp `Socket` chủ động chuyển các lỗi hệ thống về exception ở mức C++ (`SocketException`, `SocketTimeout`), giúp code xử lý luồng điều khiển rõ ràng hơn (đặc biệt trong vòng lặp nhận RTP).

```
1 int Socket::send(const uint8_t* data, size_t length) {
2     size_t totalSent = 0;
3     while (totalSent < length) {
4         int bytes = ::send(sockfd_, (const char*)(data + totalSent),
5                             length - totalSent, 0);
6         if (bytes == SOCKET_ERROR) {
7             // map error -> SocketTimeout / SocketException
8             ...
9         }
10        totalSent += bytes;
11    }
12    return (int)totalSent;
13 }
```

Listing 2: Vòng lặp `send()` đảm bảo gửi đủ dữ liệu RTSP qua TCP

Với RTP (UDP), socket được đặt timeout ngắn để thread nhận không bị block vĩnh viễn và có thể thoát khi `running_` chuyển trạng thái.

```
1 socket_->setTimeout(5); // milliseconds (UDP)
```

```
2...
3try {
4    int bytes = socket->recvFrom(buffer, sizeof(buffer), senderAddr);
5    ...
6} catch (const SocketTimeout&) {
7    continue; // loop again to check running_
8}
```

Listing 3: Nhận RTP với timeout để kiểm tra cờ dừng định kỳ

2.2.2 RTSPMessage: parse/build theo định dạng dòng

Thay vì parse “thủ công” rả rác, hệ thống gom chuẩn hóa RTSP ở lớp `RTSPMessage`. Lớp này cung cấp: (1) `buildRequest/buildResponse` để sinh chuỗi RTSP chuẩn CRLF, (2) `parseRequest/parseResponse` để tách request line/status line và headers vào `std::map`, (3) quy ước đọc `CSeq` thành số nguyên để dùng trực tiếp.

```
1std::string RTSPMessage::buildRequest(const std::string& method,
2                                     const std::string url,
3                                     int cseq,
4                                     const std::map<std::string, std::string>& headers)
5{
6    std::stringstream ss;
7    ss << method << " " << url << " RTSP/1.0\r\n";
8    ss << "CSeq: " << cseq << "\r\n";
9    for (const auto& pair : headers) ss << pair.first << ": " << pair.second << "\r\n";
10    ss << "\r\n";
11    return ss.str();
12}
```

Listing 4: Sinh request RTSP chuẩn CRLF và header CSeq

2.2.3 RTPPacket và xử lý wrap-around sequence number

`RTPPacket` hiện thực header RTP (version, marker, payload type, sequence number, timestamp, SSRC) và payload. Một điểm dễ sai trong RTP là **sequence number 16-bit** sẽ wrap-around;

vì vậy project bổ sung hàm `sequenceDifference()` để đo khoảng cách sequence theo modulo 2^{16} , phục vụ thống kê mất gói ở Client và kiểm soát thứ tự ở Reassembler.

```
1 int32_t RTPPacket::sequenceDifference(uint16_t seq1, uint16_t seq2) {
2     int32_t diff = (int32_t)seq1 - (int32_t)seq2;
3     if (diff > 32768) diff -= 65536;
4     if (diff < -32768) diff += 65536;
5     return diff;
6 }
```

Listing 5: Tính chênh lệch sequence theo modulo để xử lý wrap-around

2.2.4 Builder pattern cho RTPPacket

Để giảm lỗi khi dựng packet và tăng tính “self-documenting”, hệ thống sử dụng Builder pattern: `RTPPacketBuilder` yêu cầu set các trường bắt buộc (`sequenceNumber/timestamp/ssrc/payload`) trước khi `build()`. Nếu thiếu trường, builder ném `std::runtime_error` để fail-fast ngay tại nơi tạo packet.

```
1 RTPPacket RTPPacketBuilder::build() {
2     if (!seqSet_) throw std::runtime_error("Sequence number not set");
3     if (!timestampSet_) throw std::runtime_error("Timestamp not set");
4     if (!ssrcSet_) throw std::runtime_error("SSRC not set");
5     if (!payloadSet_) throw std::runtime_error("Payload not set");
6     RTPPacket packet;
7     packet.setSequenceNumber(sequenceNumber_);
8     packet.setTimestamp(timestamp_);
9     packet.setSSRC(ssrc_);
10    packet.setPayload(payload_);
11    packet.encode(); // serialize header -> raw bytes
12    return packet;
13 }
```

Listing 6: Builder kiểm tra các trường bắt buộc trước khi build RTP packet

2.3 Server: RTSP state machine và luồng streaming RTP

2.3.1 Mô hình đa luồng và quản lý session

`RTSPServer` chạy vòng lặp `accept()` trên TCP listen socket. Mỗi kết nối mới được gán `clientId`, tạo một `ServerWorker` và chạy nó trên một `std::thread` riêng. Danh sách session được bảo vệ bởi `std::mutex` để tránh race khi stop server.

```
1 while (running) {
2     std::unique_ptr<Socket> clientSocket = listenSocket->accept();
3     ClientSession session;
4     session.clientId = nextClientId++;
5     session.worker = std::make_unique<ServerWorker>(session.clientId,
6         std::move(clientSocket));
7     session.workerThread = std::thread(&ServerWorker::run, session.worker.get());
8     activeSession.push_back(std::move(session));
9 }
```

Listing 7: Tạo thread cho từng client trong `RTSPServer::run()`

2.3.2 ServerWorker: RTSP state machine và phân phối handler

Mỗi `ServerWorker` duy trì state nội bộ `INIT/READY/PLAYING/PAUSED`. Vòng lặp `handleRtspRequests()` nhận request RTSP, parse, sau đó dispatch sang `handleSetup/handlePlay/handlePause/handleTeardown`. Việc tách handler giúp mỗi lệnh RTSP có phạm vi trách nhiệm rõ ràng (validate session, đổi state, start/stop thread streaming).

```
1 auto request = RTSPMessage::parseRequest(msg);
2 if (request.method == "SETUP") response = handleSetup(request);
3 else if (request.method == "PLAY") response = handlePlay(request);
4 else if (request.method == "PAUSE") response = handlePause(request);
5 else if (request.method == "TEARDOWN") response = handleTeardown(request);
6 else response = RTSPMessage::buildResponse(400, "Bad Request", request.cseq);
```

Listing 8: Dispatch lệnh RTSP theo method ở `ServerWorker`

2.3.3 Streaming loop: điều phối FPS, đọc frame, encode và gửi UDP

Khi nhận PLAY hợp lệ, Worker khởi động một background thread chạy `streamingLoop()`. Vòng lặp này có ba bước trọng tâm: (1) đọc frame từ `VideoStream`, (2) encode frame thành một hoặc nhiều RTP packet (Strategy), (3) gửi packet qua UDP tới `clientIP_ : clientRTPPort_`. `Timer` được dùng để kiểm soát nhịp gửi theo interval.

```
1 while (streaming_) {
2     frameTimer_>start();
3
4     std::vector<uint8_t> frame = videoStream_->nextFrame();
5     auto packets = encodingStrategy_->encode(frame, sequenceNumber_, timestamp_, ssrc_);
6
7     for (auto& packet : packets) {
8         socket_->sendTo(packet.getPacketVector().data(), packet.getLength(),
9                         clientIP_, clientRTPPort_);
10        sequenceNumber_ = packet.getSequenceNumber() + 1;
11    }
12
13    timestamp_ += 3600;    // advance timestamp per frame (project convention)
14    frameTimer_->wait();
15 }
```

Listing 9: Luồng streaming: đọc frame, encode RTP và gửi UDP

2.3.4 Strategy pattern: SD vs HD và phân mảnh payload

Với video MJPEG, kích thước frame có thể vượt MTU. Để tránh UDP fragmentation ở tầng IP, hệ thống giới hạn `MAX_PAYLOAD_SIZE` (1400 bytes) và tự phân mảnh ở tầng RTP. Strategy pattern được dùng để tách logic encode: `SEncodingStrategy` gửi một packet/1 frame (chỉ phù hợp frame nhỏ), còn `HDEncodingStrategy` chia frame thành nhiều packet, dùng marker bit để đánh dấu fragment cuối.

```
1 while (offset < frameData.size()) {
2     size_t chunk = std::min((size_t)RTPPacket::MAX_PAYLOAD_SIZE,
3                             frameData.size() - offset);
4     bool isLast = (offset + chunk >= frameData.size());
```

```
5
6   RTPPacket pkt = RTPPacketBuilder()
7       .setSequenceNumber(seqNum++)
8       .setTimestamp(timestamp)
9       .setSSRC(ssrc)
10      .setMarker(isLast ? 1 : 0)
11      .setPayload(&frameData[offset], chunk)
12      .build();
13
14  packets.push_back(pkt);
15  offset += chunk;
16 }
```

Listing 10: HD encoding: chia frame thành nhiều RTP packet và set marker cho fragment cuối

2.3.5 VideoStream: thuật toán trích xuất frame MJPEG bằng marker JPEG

File `.Mjpeg` trong project là chuỗi các ảnh JPEG nối tiếp. Để trích xuất frame, `VideoStream` đọc file vào buffer và tìm cặp marker JPEG: SOI `0xFF 0xD8` (Start Of Image) và EOI `0xFF 0xD9` (End Of Image). Khi tìm được EOI, lớp này cắt đoạn byte tương ứng và trả về như một frame hoàn chỉnh.

```
1 while (file_.read((char*)&byte, 1)) {
2     buffer_.push_back(byte);
3
4     if (buffer_.size() >= 2) {
5         uint8_t b1 = buffer_[buffer_.size()-2];
6         uint8_t b2 = buffer_[buffer_.size()-1];
7
8         if (!inFrame && b1 == 0xFF && b2 == 0xD8) {
9             inFrame = true; frameStart = buffer_.size() - 2;
10        }
11        if (inFrame && b1 == 0xFF && b2 == 0xD9) {
12            size_t frameEnd = buffer_.size();
13            std::vector<uint8_t> frame(buffer_.begin()+frameStart,
14                                     buffer_.begin()+frameEnd);
```

```
14     buffer_.erase(buffer_.begin(), buffer_.begin()+frameEnd);  
15     return frame;  
16 }  
17 }  
18 }
```

Listing 11: Quét marker JPEG SOI/EOI để cắt frame trong `VideoStream::nextFrame()`

2.4 Client: RTSP điều khiển, nhận RTP, ghép frame và buffer/cache

2.4.1 RTSPClient: phiên điều khiển và cơ chế “restart”

`RTSPClient` chịu trách nhiệm tạo kết nối TCP đến server, gửi các request RTSP (SETUP/-PLAY/PAUSE/TEARDOWN), và parse response để lấy `Session` ID. Với yêu cầu “replay” sau khi hết video, client hỗ trợ cơ chế restart bằng header `Restart: true` trong PLAY; server kiểm tra header này để reset `VideoStream` về đầu file.

```
1 std::map<std::string, std::string> headers;  
2 headers["Session"] = sessionId_;  
3 if (restart) headers["Restart"] = "true";  
4  
5 std::string request = RTSPMessage::buildRequest("PLAY", rtspUri_, cseq_, headers);  
6 socket_->send((const uint8_t*)request.c_str(), request.size());
```

Listing 12: Client gửi PLAY kèm header Restart khi cần phát lại

2.4.2 RTPReceiver: nhận UDP và thống kê mất gói

`RTPReceiver` chạy background thread `receiveLoop()` để nhận packet và chuyển cho `FrameReassembler`. Đồng thời, receiver cập nhật thống kê loss dựa trên chênh lệch sequence; phần này dựa trực tiếp trên `RTPPacket::sequenceDifference()` để xử lý wrap-around.

```
1 int32_t diff = RTPPacket::sequenceDifference(currentSeq, lastSequenceNumber_);  
2 if (diff > 1) {  
3     packetLost_ += static_cast<uint64_t>(diff - 1);  
4 }  
5 lastSequenceNumber_ = currentSeq;
```


Listing 13: Tính packet loss dựa trên chênh lệch sequence

2.4.3 FrameReassembler: ghép fragment theo timestamp và marker

Đối với HD mode, một frame có thể gồm nhiều RTP packet. **FrameReassembler** gom các fragment theo **timestamp**, sắp thứ tự theo **sequenceNumber**, và xác định điểm kết thúc frame dựa trên marker bit. Khi frame hoàn chỉnh, reassembler nối payload và push vào **FrameBuffer**. Để tránh leak khi mất marker (mất gói), module cũng hỗ trợ giới hạn bộ nhớ cho map và chiến lược “flush” khi đổi timestamp.

```
1 if (packet.getMarker() == 1) {
2     auto& fragments = frameFragments_[timestamp];
3     std::sort(fragments.begin(), fragments.end(),
4               [](auto& a, auto& b){ return a.getSequenceNumber() <
5                 b.getSequenceNumber(); });
6
7     std::vector<uint8_t> frameData;
8     for (auto& frag : fragments) {
9         const auto& payload = frag.getPayload();
10        frameData.insert(frameData.end(), payload.begin(), payload.end());
11    }
12
13    frameBuffer_->push(frameData);
14    frameFragments_.erase(timestamp);
15 }
```

Listing 14: Ghép frame khi gặp marker bit và đẩy vào buffer

2.4.4 FrameBuffer: buffer như một “video cache” thread-safe

FrameBuffer là lớp trung gian quan trọng nhất để “hấp thụ jitter” mạng. Nó được hiện thực như một hàng đợi thread-safe, với **push()** từ thread nhận RTP và **tryPop()** từ UI thread. **std::mutex** và **std::condition_variable** được dùng để đồng bộ; **close()** đảm bảo thread consumer không bị kẹt khi shutdown.

```
1 void FrameBuffer::push(const std::vector<uint8_t>& frame) {  
2     std::unique_lock<std::mutex> lock(mtx_);  
3     if (closed_) return;  
4     buffer_.push_back(frame);  
5     cv_.notify_one();  
6 }  
7  
8 bool FrameBuffer::tryPop(std::vector<uint8_t>& frame) {  
9     std::unique_lock<std::mutex> lock(mtx_);  
10    if (buffer_.empty()) return false;  
11    frame = std::move(buffer_.front());  
12    buffer_.pop_front();  
13    return true;  
14 }
```

Listing 15: FrameBuffer đồng bộ bằng mutex + condition variable

2.4.5 ClientUI (Qt): prebuffer, adaptive playback và timeline buffered

Ở phía UI, state machine được đơn giản hóa thành INIT/READY/PLAYING. Khi PLAY, UI không render ngay lập tức mà đợi buffer đạt ngưỡng PREBUFFER_FRAMES; đây là cơ chế prebuffer nhằm tạo “độ trễ chủ động” khoảng vài giây để chống đứt quãng. Đồng thời, UI điều chỉnh interval của frameTimer_ theo kích thước buffer để tránh tiêu thụ hết buffer khi mạng chậm.

```
1 if (!prebufferReady_) {  
2     size_t bufferSize = frameBuffer_->size();  
3     if (bufferSize >= PREBUFFER_FRAMES) {  
4         prebufferReady_ = true;  
5     } else {  
6         statusLabel_->setText(QString("Buffering... %1/%2 frames")  
7                                 .arg(bufferSize).arg(PREBUFFER_FRAMES));  
8         return;  
9     }  
10 }
```

Listing 16: Prebuffer: chỉ phát khi buffer đủ PREBUFFER_FRAMES

```
1 if (currentBufferSize >= 20)      frameTimer_>setInterval(38);  
2 else if (currentBufferSize >= 10) frameTimer_>setInterval(40);  
3 else if (currentBufferSize >= 5)  frameTimer_>setInterval(45);  
4 else                             frameTimer_>setInterval(50);
```

Listing 17: Adaptive timer: tăng interval khi buffer thấp để giảm giật

Timeline slider được mở rộng thành `BufferedSlider` để hiển thị vùng đã buffer (“buffered position”), tính bằng `currentFrame + bufferSize`. Phần vẽ bổ sung được hiện thực trong `paintEvent()`.

```
1 int bufferedPos = (bufferedValue_ - minimum()) * groove.width() / total;  
2 if (bufferedPos > currentPos) {  
3     QRect bufferedRect = groove;  
4     bufferedRect.setLeft(groove.left() + currentPos);  
5     bufferedRect.setRight(groove.left() + bufferedPos);  
6     painter.fillRect(bufferedRect, QColor(180, 180, 180, 200));  
7 }
```

Listing 18: Vẽ vùng buffered trên timeline slider

2.5 Ghi nhận triển khai: logging, cấu hình và khả năng mở rộng

Hệ thống sử dụng `Logger` (Singleton) để log đồng thời ra console và file, đảm bảo thread-safe bằng `std::mutex`. Việc cấu hình tách riêng ở `Config` (SingletonWithInit) cho phép thay đổi tham số runtime (port, video file, log level) mà không cần rebuild.

```
1 void Logger::log(LogLevel level, const std::string& message) {  
2     std::lock_guard<std::mutex> lock(mutex_);  
3     if (!initialized_ || level < minLevel_) return;  
4     std::cout << "[" << getCurrentTime() << " ] [" << levelToString(level) << " ] "  
5         << message << std::endl;  
6     if (logFile_.is_open()) logFile_ << ...;  
7 }
```

Listing 19: Logger Singleton: log thread-safe và lọc theo mức



Về khả năng mở rộng, các điểm đã “cắm sẵn” trong thiết kế gồm: Strategy cho encoding (thêm H.264/H.265 packetization), Builder cho các message phức tạp hơn, và RTSP state machine tách handler để bổ sung DESCRIBE/SET_PARAMETER/RANGE trong tương lai mà không làm vỡ luồng chính.

3 Results

Phần này tổng hợp kết quả kiểm thử và xác nhận tính đúng đắn của các khối logic cốt lõi (RTSP message handling, Timer kiểm soát nhịp), đồng thời mô tả các hành vi runtime có thể quan sát được từ client UI và các thống kê có sẵn trong mã nguồn.

3.1 Kết quả kiểm thử đơn vị (Unit Tests)

Dự án cung cấp bộ kiểm thử trong `common/tests` cho hai thành phần có rủi ro lỗi cao: (1) xây dựng và phân tích RTSP message, (2) Timer điều phối thời gian. Các test được viết theo phong cách “lightweight framework” (assert + reporter) và có thể chạy độc lập với Qt.

Table 5: Tổng hợp kết quả unit test ở tầng Common

Test Suite	Số test	Kết quả
<code>test_rtsp_message</code>	15	15 passed, 0 failed
<code>test_timer</code>	13	13 passed, 0 failed

Kết quả chạy test RTSPMessage thể hiện cả các kịch bản cơ bản lẫn round-trip (build rồi parse lại). Đoạn log dưới đây minh họa phần tổng kết của suite (trích nguyên văn từ output).

```
1 Total: 15 tests
2 Passed: 15
3 Failed: 0
```

Listing 20: Output tổng kết của `test_rtsp_message`

Tương tự, Timer được kiểm tra với nhiều interval (25 FPS, 30 FPS, 60 FPS), kiểm tra drift (không sleep khi “work time” vượt interval), và stress test. Đoạn log tổng kết:

```
1 Total: 13 tests
2 Passed: 13
3 Failed: 0
```

Listing 21: Output tổng kết của `test_timer`

3.2 Chỉ số runtime sẵn có ở Client

Ngoài unit test, Client đã tích hợp các thống kê runtime cho RTP: số packet nhận, số packet ước lượng mất, tổng bytes, và phần trăm loss. Loss được tính bằng chênh lệch sequence; do đó thống kê phản ánh mất gói “theo thứ tự” (out-of-order có thể bị tính như lost nếu đến quá muộn hoặc bị bỏ qua ở Reassembler tùy kịch bản).

```
1 double RTPReceiver::getPacketLossPercentage() const {  
2     uint64_t total = packetReceived_ + packetLost_;  
3     if (total == 0) return 0.0;  
4     return (static_cast<double>(packetLost_) * 100.0) / static_cast<double>(total);  
5 }
```

Listing 22: Công thức phần trăm mất gói RTP

3.3 Hành vi buffer và prebuffer quan sát từ UI

UI triển khai hai cơ chế nhằm tối ưu trải nghiệm: (1) prebuffer trước khi phát, (2) điều chỉnh nhịp render theo mức buffer. Prebuffer được kích hoạt khi bắt đầu PLAY hoặc khi xảy ra “rebuffering” do buffer cạn; điều này làm tăng độ ổn định, đổi lại là tăng độ trễ đầu phát.

Table 6: Các tham số buffer quan trọng và ý nghĩa vận hành

Tham số	Vị trí	Ý nghĩa
PREBUFFER_FRAMES	ClientUI	Số frame cần tích lũy trước khi bắt đầu render
LOW_BUFFER_THRESHOLD	ClientUI	Ngưỡng cảnh báo buffer thấp và tăng interval render
MAX_PAYLOAD_SIZE	RTPPacket	Giới hạn payload/packet để tránh IP fragmentation

```
1 statusLabel_->setText(QString("Buffering... %1/%2 frames (%3%)")  
2     .arg(bufferSize)  
3     .arg(PREBUFFER_FRAMES)  
4     .arg(percentage, 0, 'f', 1));
```

Listing 23: UI chuyển sang trạng thái “Buffering” khi buffer chưa đủ

3.4 Hướng dẫn tái lập kết quả (Reproducibility)

Để tái lập kiểm thử và quan sát hệ thống trong môi trường có Qt6, quy trình thực nghiệm gồm: build project, chạy server với port, chạy client với tham số (server IP, port, file MJPEG, RTP port). Các lệnh chạy và tham số được đọc trực tiếp từ `server/src/server.cpp` và `client/src/client.cpp`.

```
1. ./server 8554
2. ./client 127.0.0.1 8554 movie.Mjpeg 25000
```

Listing 24: Gợi ý chạy server/client theo tham số dòng lệnh

Trong quá trình chạy, file log `server.log` và `client.log` sẽ ghi lại các sự kiện chính (kết nối, chuyển state, buffer cảnh báo, timeout), phục vụ phân tích sau thực nghiệm.

4 Conclusion

Hệ thống đã hiện thực đầy đủ một pipeline streaming video MJPEG theo mô hình RTSP (control plane, TCP) và RTP (data plane, UDP), với cấu trúc module hóa rõ ràng (Common/Server/-Client), cơ chế đa luồng để phục vụ đồng thời nhiều client, và cơ chế buffer/prebuffer ở phía client để ổn định playback khi mạng jitter.

4.1 Tổng kết đóng góp kỹ thuật theo mã nguồn

Đóng góp cốt lõi của project không chỉ nằm ở việc “chạy được streaming”, mà ở các quyết định hiện thực giúp hệ thống dễ bảo trì và mở rộng: chuẩn hóa RTSP ở `RTSPMessage`, chuẩn hóa networking ở `Socket`, áp dụng Builder cho RTP packet để kiểm soát tính hợp lệ, Strategy cho encode để tách SD/HD và phân mảnh.

```
1 RTSPClient::connect() -> Socket(TCP)::connect(serverIP, serverPort);  
2 RTPReceiver::start() -> Socket(UDP)::bind("0.0.0.0", clientRTPPort);  
3 ServerWorker::handleSetup() -> parse Transport header (client_port=...)  
4 ServerWorker::streamingLoop() -> Socket(UDP)::sendTo(..., clientIP, clientRTPPort);
```

Listing 25: Tách biệt control plane và data plane: RTSP qua TCP, RTP qua UDP

Ngoài ra, lớp `FrameBuffer` có thể xem là một dạng “video cache” in-memory: nó lưu các frame đã ghép, tách biệt nhịp nhận (network-driven) khỏi nhịp phát (UI-driven). Đây là nền tảng để hiện thực prebuffer và các chính sách điều chỉnh nhịp phát theo mức buffer.

```
1 /* Producer thread */  
2 frameReassembler_ -> frameBuffer() -> push(frameData);  
3  
4 /* Consumer (UI timer) */  
5 std::vector<uint8_t> frame;  
6 if (frameBuffer_ -> tryPop(frame)) { render(frame); }
```

Listing 26: Buffer như cache: producer (RTP) và consumer (UI) tách nhịp

4.2 Hạn chế hiện tại và đề xuất cải tiến dựa trên bằng chứng mã nguồn

Một số điểm trong code hiện tại có thể ảnh hưởng độ ổn định hoặc độ chính xác timing; các đề xuất dưới đây được nêu dựa trực tiếp trên các đoạn mã hiện có.

4.2.0.1 (1) Tham số Timer ở Server chưa khớp mục tiêu 25 FPS. Trong `ServerWorker` có chú thích “25 FPS (40ms)” nhưng interval đang được khởi tạo là 10ms, dẫn tới nhịp gửi nhanh hơn dự kiến và có thể làm tăng tải CPU/network.

```
1 // Khi to timer cho 25 FPS (40ms)
2 frameTimer_ = std::make_unique<Timer>(10);
```

Listing 27: Interval Timer ở `ServerWorker` đang đặt 10ms dù comment nói 40ms

Khuyến nghị: đưa interval về 40ms hoặc đọc từ `Config` để đồng bộ với FPS mục tiêu và dễ hiệu chỉnh theo video.

4.2.0.2 (2) Logic stop() ở ServerWorker dùng static flag có thể gây lỗi khi có nhiều worker. Hàm `stop()` đang dùng `static std::atomic<bool> stopCalled` khiến chỉ worker đầu tiên được stop “thực sự”, các worker còn lại sẽ return sớm. Đây là rủi ro rõ ràng nếu server phục vụ nhiều client đồng thời.

```
1 void ServerWorker::stop() {
2     static std::atomic<bool> stopCalled{false};
3     if (stopCalled.exchange(true)) {
4         return;
5     }
6     // ...
7 }
8
```

Listing 28: Static stop flag có thể làm dừng sai khi nhiều worker

Khuyến nghị: biến `stopCalled` thành member per-instance hoặc bỏ hoàn toàn (vì bản thân `running_` và `streaming_` đã là cờ dừng).

4.2.0.3 (3) Tính năng tua (seek) trên timeline hiện mới là thay đổi chỉ số hiển thị, chưa có RTSP Range. Client cập nhật `currentFrame_` theo slider nhưng không gửi yêu cầu RTSP mới để server bắt đầu từ vị trí tương ứng; do đó “seek” hiện tại chỉ mang tính UI/telemetry.

```
1 void ClientUI::onTimelineSliderReleased() {
2     int targetFrame = timelineSlider_>value();
```

```
3     currentFrame_ = targetFrame;  
4 }
```

Listing 29: Seek hiện tại chỉ cập nhật currentFrame_ ở UI

Khuyến nghị: mở rộng RTSP với header Range (NPT hoặc frame index), và server cần hỗ trợ reset `VideoStream` đến offset phù hợp (có thể bằng index table hoặc scan marker nhanh).

4.2.0.4 (4) Một số chi tiết định dạng RTSP request nên được chuẩn hóa tuyệt đối. Ví dụ, trong `RTSPClient::sendTeardown()` dòng request line thiếu khoảng trắng trước `RTSP/1.0`. Đây là lỗi định dạng có thể gây fail khi tương tác với server khác.

```
1 // Sai:  
2 ss << "TEARDOWN " << rtspUri_ << "RTSP/1.0\r\n";  
3 // ng   phi   l :  
4 ss << "TEARDOWN " << rtspUri_ << " RTSP/1.0\r\n";
```

Listing 30: Teardown request line thiếu khoảng trắng trước phiên bản

Khuyến nghị: thống nhất tất cả request thông qua `RTSPMessage::buildRequest()` để loại bỏ sai sót format.

4.3 Kết luận

Với các thành phần đã hiện thực, dự án đáp ứng yêu cầu nền tảng của một hệ thống streaming RTSP/RTP có khả năng phân mảnh frame lớn, quản lý session, và buffer hóa ở client. Đồng thời, do code đã có cấu trúc pattern rõ ràng (Builder/Strategy/Singleton) và ranh giới module sạch, hệ thống sẵn sàng để mở rộng sang các tính năng “production-grade” hơn như Range seek, codec khác, adaptive bitrate, và cơ chế đồng bộ A/V trong các phiên bản tiếp theo.



List of Figures



List of Tables